

Streams · Redirection · Piping | sort · cut · paste

Linux Administration 24CSE0213 | Chitkara University | Lectures 37–42

1. Standard Streams

Every Linux process has 3 built-in data channels called streams:

Stream	fd	Default	Purpose
stdin	0	Keyboard	Where program reads input from
stdout	1	Screen	Where program prints normal output
stderr	2	Screen	Where program prints error messages

stdout and stderr both appear on screen by default — they look the same. Redirection lets you separate them.

2. Redirection > >> < 2>

Operator	What it does	Example
>	Redirect stdout to file (OVERWRITES)	ls -la > list.txt
>>	Redirect stdout, APPEND to file	date >> log.txt
<	Feed file as input (stdin)	sort < names.txt
2>	Redirect stderr to file	find /etc 2> err.txt
2>&1	Merge stderr into stdout stream	cmd > out.txt 2>&1
&>	Redirect BOTH stdout+stderr (bash shorthand)	cmd &> all.txt
2>/dev/null	Discard errors silently	find / 2>/dev/null

⚠ > **DESTROYS** the file. Always double-check before using it on important files.

⚠ Order matters! cmd 2>&1 > file is **WRONG**. Correct: cmd > file 2>&1

2.1 Quick Examples

```
echo 'Hello' > greet.txt           # create file
date >> greet.txt                  # append — now file has 2 lines
wc -l < /etc/passwd               # feed passwd as input, count lines
find /etc -name '*.conf' > found.txt 2> errors.txt # split streams
ls /home /fakedir > out.txt 2>&1  # capture EVERYTHING to one file
```

TASK 1 Easy

Run: ls /home /fakedir /etc/passwd Then redo it so: a) Only errors go to ~/err.txt b) Both stdout+stderr go to ~/all.txt

✓ Solution — Task 1

```
ls /home /fakedir /etc/passwd 2> ~/err.txt # a) errors only
ls /home /fakedir /etc/passwd &> ~/all.txt  # b) everything
cat ~/err.txt → ls: cannot access '/fakedir': No such file or directory
cat ~/all.txt → shows error + /etc/passwd + /home listing
```

3. Pipes |

A pipe connects stdout of the LEFT command to stdin of the RIGHT command. Data flows left → right through the pipeline. No temporary files needed.

```
command1 | command2 | command3 # chain as many as you need
```

3.1 Common Pipe Patterns

Pipeline	What it does
<code>ls -la less</code>	Scroll through long output
<code>cat /etc/passwd wc -l</code>	Count total user accounts
<code>ps aux grep firefox</code>	Find if firefox is running
<code>history grep apt tail -10</code>	Last 10 apt commands you ran
<code>cat /etc/passwd grep '/bin/bash'</code>	Users whose shell is bash
<code>du -sh /var/* sort -h tail -5</code>	5 biggest dirs in /var
<code>cmd tee file.txt wc -l</code>	Save output AND count lines simultaneously

3.2 tee — See output AND save it

```
ps aux | tee ~/procs.txt | grep nginx # saves all, shows only nginx
sudo apt update 2>&1 | tee update.log # watch + log at same time
```

TASK 2 Easy

Using ONE pipeline (no temp files): a) Count how many accounts in /etc/passwd use /bin/bash as shell
b) Show the top 3 memory-using processes (column 4 is %MEM)

✓ Solution — Task 2

```
cat /etc/passwd | grep '/bin/bash' | wc -l
2
ps aux | sort -k4 -rn | head -4 | tail -3
student 1234 2.5 8.1 ... firefox
root 5678 0.0 1.3 ... Xorg
student 9012 0.5 0.9 ... python3
```

4. sort

Sorts lines of text alphabetically or numerically. Used constantly in pipelines.

Option	Effect	Example
<code>sort file</code>	A–Z alphabetical (default)	<code>sort names.txt</code>
<code>-r</code>	Reverse order (Z–A or high→low)	<code>sort -r file.txt</code>
<code>-n</code>	Numeric order (1,2,10 not 1,10,2)	<code>sort -n nums.txt</code>
<code>-rn</code>	Numeric reverse (biggest first)	<code>sort -rn -k3</code>
<code>-k N</code>	Sort by column N	<code>sort -k2 data.txt</code>
<code>-t ':'</code>	Set delimiter for -k	<code>sort -t: -k3 -n /etc/passwd</code>
<code>-u</code>	Remove duplicate lines	<code>sort -u file.txt</code>
<code>-h</code>	Human sizes (1K 2M 3G)	<code>du -sh * sort -h</code>

```
sort -t: -k3 -n /etc/passwd | head -3 # sort users by UID
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

```
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

```
du -sh /var/* 2>/dev/null | sort -h      # disk usage, smallest first
4.0K  /var/mail      40K  /var/cache      2.3M  /var/lib      12M  /var/log
```

TASK 3

Medium

Create ~/scores.txt with: Alice 85 / Bob 42 / Carol 91 / Dave 73 / Eve 58 (one per line) a) Sort by score, highest first b) Show ONLY the name of the top scorer

✓ Solution — Task 3

```
printf 'Alice 85\nBob 42\nCarol 91\nDave 73\nEve 58\n' > ~/scores.txt
sort -k2 -rn ~/scores.txt
Carol 91 | Alice 85 | Dave 73 | Eve 58 | Bob 42
sort -k2 -rn ~/scores.txt | head -1 | cut -d' ' -f1
Carol
```

5. cut — Extract Columns

cut slices out specific fields or characters from each line of input. Essential for structured text like CSV, /etc/passwd, log files.

Option	Meaning	Example
-d ' , '	Set delimiter (default is TAB)	cut -d, -f2 file.csv
-f N	Extract field number N	cut -f1 → first column
-f 1,3	Fields 1 AND 3	cut -d: -f1,3 /etc/passwd
-f 2-	Field 2 through end of line	Skips first column
-f 1-3	Fields 1 through 3	First three columns
-c 1-5	Characters 1 to 5	First 5 chars per line

5.1 cut Examples

```
cut -d: -f1 /etc/passwd          # all usernames
cut -d: -f1,7 /etc/passwd       # username + shell
cut -d: -f6 /etc/passwd | sort -u # unique home directories
```

CSV example — create the test file once, use for all cut/paste tasks:

```
cat > ~/emp.csv << EOF
Name,Dept,Salary,City
Alice,Engineering,75000,Delhi
Bob,Marketing,55000,Mumbai
Carol,Engineering,82000,Bangalore
EOF
```

```
cut -d, -f1 ~/emp.csv          # names only
Name / Alice / Bob / Carol
cut -d, -f1,4 ~/emp.csv        # name + city
Name,City / Alice,Delhi / Bob,Mumbai / Carol,Bangalore
cut -d, -f2 ~/emp.csv | sort | uniq -c # count per dept
2 Engineering 1 Dept 1 Marketing
```

TASK 4 Medium

Using ~/emp.csv: a) Show ONLY names of Engineering employees (no header) b) Find the highest salary — display as: Top Salary: 82000

✓ Solution — Task 4

```
grep 'Engineering' ~/emp.csv | cut -d, -f1
Alice
Carol
tail -n +2 ~/emp.csv | cut -d, -f3 | sort -n | tail -1 | xargs -I{} echo 'Top Salary: {}'
Top Salary: 82000
🔗 tail -n +2 skips the header row (start from line 2 onwards)
```

6. paste — Merge Files Side by Side

paste joins files horizontally — line 1 of file A + line 1 of file B, and so on. Think of it as the reverse of cut.

Option	Meaning	Example
paste f1 f2	Merge line-by-line, TAB separator	paste names.txt ages.txt
-d ' , '	Use comma instead of TAB	paste -d, f1 f2

-d ' '	Use pipe char as separator	paste -d' ' f1 f2
-s	Serial: all lines of one file in a row	paste -s names.txt
-s -d ','	All lines as comma-separated list	Useful for config values

6.1 paste Examples

```
printf 'Alice\nBob\nCarol\n' > ~/names.txt
printf 'Delhi\nMumbai\nBangalore\n' > ~/cities.txt
```

```
paste ~/names.txt ~/cities.txt          # TAB-separated
```

```
Alice    Delhi
Bob       Mumbai
Carol     Bangalore
```

```
paste -d, ~/names.txt ~/cities.txt      # comma-separated
```

```
Alice,Delhi / Bob,Mumbai / Carol,Bangalore
```

```
paste -s -d, ~/names.txt                # all names on ONE line
```

```
Alice,Bob,Carol
```

TASK 5 Easy

Create ~/products.txt (Laptop/Phone/Tablet) and ~/prices.txt (55000/20000/35000) a) Merge with comma separator — display and save to ~/catalog.csv b) Put all product names on ONE comma-separated line

✓ Solution — Task 5

```
printf 'Laptop\nPhone\nTablet\n' > ~/products.txt
printf '55000\n20000\n35000\n' > ~/prices.txt
paste -d, ~/products.txt ~/prices.txt | tee ~/catalog.csv
Laptop,55000 / Phone,20000 / Tablet,35000
paste -s -d, ~/products.txt
Laptop,Phone,Tablet
```

7. Putting It All Together

7.1 Real-World One-Liners

```
# Sorted list of all usernames on the system
cut -d: -f1 /etc/passwd | sort
```

```
# Count files by extension in /etc
ls /etc | grep '\.' | cut -d. -f2 | sort | uniq -c | sort -rn
```

```
# Top 5 IPs with failed SSH logins
sudo grep 'Failed password' /var/log/auth.log | cut -d' ' -f11 | sort | uniq -c | sort -rn | head -5
```

```
# Get all unique login shells used on this system
cut -d: -f7 /etc/passwd | sort -u
```

```
# Show process count per user
ps aux | tail -n +2 | awk '{print $1}' | sort | uniq -c | sort -rn
```

7.2 Final Challenge

TASK 6 Hard

Using ONLY the terminal (no editors), create ~/report.txt containing: Line 1: === SYSTEM REPORT
=== Line 2: current date Line 3: logged-in user Line 4: total user accounts in /etc/passwd Line 5:
name of the process using most CPU right now Line 6: === END === Then display the file.

✓ Solution — Task 6

```
echo '=== SYSTEM REPORT ===' > ~/report.txt
date >> ~/report.txt
echo "User: $(whoami)" >> ~/report.txt
echo "Total accounts: $(wc -l < /etc/passwd)" >> ~/report.txt
ps aux | sort -k3 -rn | head -2 | tail -1 | awk '{print "Top CPU: "$11}' >> ~/report.txt
echo '=== END ===' >> ~/report.txt
cat ~/report.txt

=== SYSTEM REPORT ===
Mon Mar 16 09:30:00 IST 2026
User: student
Total accounts: 44
Top CPU: /usr/lib/firefox/firefox
=== END ===
```

8. Master Quick Reference Sheet

8.1 All Redirection Symbols

Symbol	Meaning	Remember
>	Redirect stdout → file (overwrite)	DESTROYS existing content!
>>	Redirect stdout → file (append)	SAFE — adds to the end
<	File → stdin of command	Feed a file as input
2>	Redirect stderr → file	2 = file descriptor of stderr
2>>	Append stderr → file	Keeps existing errors
2>&1	Merge stderr into stdout	Must come AFTER > filename
&>	Both stdout+stderr → file	Bash shorthand for > f 2>&1
2>/dev/null	Discard errors	/dev/null = black hole
	Pipe stdout → stdin of next cmd	Chain unlimited commands
tee file	Pipe AND save to file simultaneously	See + save at same time

8.2 sort Quick Reference

Flag	Use it when...
-n	Sorting numbers (without -n, 10 comes before 2)
-r	You want biggest/last first
-k N	Data has multiple columns — specify which one
-t ':'	Columns are separated by : (like /etc/passwd)
-u	You want duplicate lines removed
-h	Sizes are like 4K, 2M, 3G (du output)

8.3 cut Quick Reference

Flag	Use it when...
-d ','	Your data is CSV — always pair with -f
-d ':'	Working with /etc/passwd, /etc/group etc.
-f N	Extract column number N
-f 1,3	Extract multiple specific columns
-f 2-	Keep everything from column 2 onwards
-c 1-5	Extract first 5 characters (no delimiter needed)

8.4 paste Quick Reference

Flag	Use it when...
-d ','	You want comma-separated output instead of tab
-s	You want all lines from a file on ONE row
-s -d ','	Building a comma-separated list from a column

8.5 Common Mistakes

Mistake	Fix
cmd > file (on existing file)	Use >> to append, or back up first

cmd 2>&1 > file (wrong order)	Correct: cmd > file 2>&1
sort without -n on numbers	10 sorts before 2 without -n. Always use -n for numbers
cut -f2 on a CSV file	Default delimiter is TAB. Use cut -d, -f2 for CSV
Forgetting 2>/dev/null	find / without it floods screen with permission errors